

СОФИЙСКИ УНИВЕРСИТЕТ
„СВ. КЛИМЕНТ ОХРИДСКИ“



ФАКУЛТЕТ ПО МАТЕМАТИКА
И ИНФОРМАТИКА

ДЪРЖАВЕН ИЗПИТ

ЗА ПОЛУЧАВАНЕ НА ОКС „БАКАЛАВЪР ПО КОМПЮТЪРНИ НАУКИ“

ЧАСТ I (ПРАКТИЧЕСКИ ЗАДАЧИ)

Драги абсолвенти:

- Попълнете факултетния си номер в горния десен ъгъл на всички листове.
- Пишете само на предоставените листове, без да ги разкопчавате.
- Решението на една задача трябва да бъде на същия лист, на който е и нейното условие (т.е. може да пишете отпред и отзад на листа със задачата, но не и на лист на друга задача).
- Ако имате нужда от допълнителен лист, можете да поискате от квесторите.
- На един лист не може да има едновременно и чернова, и белова.
- Черновите трябва да се маркират, като най-отгоре на листа напишете „ЧЕРНОВА“.
- Ако решението на една задача не се побира на нейния лист, трябва да поискате нов бял лист от квесторите. Той трябва да се защити с телбод към листа със задачата.
- Всеки от допълнителните листове (белова или чернова) трябва да се надпише най-отгоре с вашия факултетен номер.
- Черновите също се предават и се защитават в края на работата.
- Времето за работа по изпита е 3 часа.

Изпитната комисия ви пожелава успешна работа!

Задача 1.

Задачата да се реши на езика C++. Реализирана е система за управление на автономни роботи – наземни роботи и летящи дронове, използвани при различни типове мисии – патрулиране (Patrol), наблюдение (Surveillance), изследване (Exploration) и спасителни (Rescue).

- Всеки робот притежава уникален идентификатор и текущо ниво на батерията.
- Наземните роботи се използват за придвижване и работа върху различни типове терен. За тях се пази информация за максималното ниво на трудност на терена (terrainLevel). Дроновете се използват за въздушно наблюдение и имат максимална височина на полет (maxAltitude).
- Всяка мисия има тип и ниво на трудност. Значението на трудността зависи от типа на мисията. При Patrol тя описва трудността на терена. При Surveillance описва необходимата височина за наблюдение, а при Rescue описва сложността на спасителната операция.
- Не всички роботи могат да изпълняват всички мисии. Наземните роботи могат да участват в мисии от тип Patrol и Rescue. Дроновете могат да участват в мисии от тип Surveillance и Rescue. Даден робот може да изпълни мисия само ако неговите технически характеристики са достатъчни за съответната трудност на мисията и ако разполага с необходимия заряд на батерията.
- При изпълнение на мисия батерията на робота намалява с количество, равно на два пъти трудността на задачата.
- Мисиите могат да се възлагат на флотилии от роботи, като флотилията е отговорна за поддържането на група от работи и предаването на мисии към тях.

Да се попълнят празните места в дадения по-долу програмен фрагмент в съответствие с условието на задачата, като се осигури коректно управление на паметта за вложените обекти. За улеснение приемете, че при заделянето на обекти с new винаги има достатъчно памет.

```
enum class MissionType { Patrol, Surveillance, Exploration, Rescue };
struct Mission { _____; _____ };
class Robot {
protected:
    std::string id;
    unsigned battery;
public:
    Robot(const std::string& id, unsigned battery) : id(id), battery(battery) {}
    _____ ~Robot() _____;
    const std::string& getId() const { return id; }
    unsigned getBattery() const { return battery; }
    _____ bool canExecute(const Mission& m) const _____;
    _____ clone() const _____;
    void print(std::ostream& os) const {
        os << " [id=" << id << ", battery=" << battery << "]; }
    bool assign(const Mission& m) {
        if (!canExecute(m)) return false;
        battery -= _____;
        return true; }
};
class GroundRobot : public Robot {
    unsigned terrainLevel;
public:
    GroundRobot(const std::string& id, unsigned battery, unsigned terrainLevel)
        : _____, _____ {}
```

```
bool canExecute(const Mission& m) const {
    return battery >= 2 * m.difficulty &&
        (m.type == MissionType::Patrol || m.type == MissionType::Rescue) &&
        terrainLevel >= m.difficulty; }
    _____ clone() const { return _____; }
};

class Drone : public Robot {
    unsigned maxAltitude;
public:
    Drone(const std::string& id, unsigned battery, unsigned maxAltitude)
        : Robot(id, battery), maxAltitude(maxAltitude) {}
    bool canExecute(const Mission& m) const {
        return _____
            _____
            _____; }
    _____ clone() const { return _____; }
};

class Fleet {
    std::vector<_____> robots;
public:
    Fleet() = default;
    ~Fleet() {
        for (Robot* r : robots) _____; }
    Fleet(const Fleet& other) {
        for (const Robot* r : other.robots) robots.push_back(_____); }
    Fleet& operator=(const Fleet& other) {
        if (_____) {
            for (Robot* r : robots) delete r;
            robots.clear();
            for (const Robot* r : other.robots) robots.push_back(_____);
        }
        return _____; }
    _____ add(const Robot& r) { robots.push_back(_____); return *this; }
    void printAll(std::ostream& os) const {
        for (const Robot* r : robots) { r->print(os); os << "\n"; } }
    int assignToAll(const Mission& m) {
        int count = 0;
        for (Robot* r : robots) { if (_____) ++count; }
        return count;
    }
};

int main() {
    Fleet fleet;
    fleet.add(GroundRobot("G1", 100, 5))
        .add(Drone("D1", 100, 8))
        .printAll(std::cout);
    std::cout << fleet.assignToAll({MissionType::Rescue, 4}) << "\n";
    return 0;
}
```

Примерно решение

```
enum class MissionType { Patrol, Surveillance, Exploration, Rescue };
struct Mission { MissionType type; unsigned difficulty; };
class Robot {
protected:
    std::string id;
    unsigned battery;
public:
    Robot(const std::string& id, unsigned battery) : id(id), battery(battery) {}
    virtual ~Robot() = default;
    const std::string& getId() const { return id; }
    unsigned getBattery() const { return battery; }
    virtual bool canExecute(const Mission& m) const = 0;
    virtual Robot* clone() const = 0;
    void print(std::ostream& os) const {
        os << " [id=" << id << ", battery=" << battery << "]; }
    bool assign(const Mission& m) {
        if (!canExecute(m)) return false;
        battery -= 2 * m.difficulty;
        return true; }
};

class GroundRobot : public Robot {
    unsigned terrainLevel;
public:
    GroundRobot(const std::string& id, unsigned battery, unsigned terrainLevel)
        : Robot(id, battery), terrainLevel(terrainLevel) {}
    bool canExecute(const Mission& m) const {
        return battery >= 2 * m.difficulty &&
            (m.type == MissionType::Patrol || m.type == MissionType::Rescue) &&
            terrainLevel >= m.difficulty; }
    /* за улеснение приемете, че new успешно заделя обекта нявсякъде в програмата */
    GroundRobot* clone() const { return new GroundRobot(*this); }
};

class Drone : public Robot {
    unsigned maxAltitude;
public:
    Drone(const std::string& id, unsigned battery, unsigned maxAltitude)
        : Robot(id, battery), maxAltitude(maxAltitude) {}
    bool canExecute(const Mission& m) const {
        return battery >= 2 * m.difficulty &&
            (m.type == MissionType::Surveillance || m.type == MissionType::Rescue) &&
            maxAltitude >= m.difficulty; }
    Drone* clone() const { return new Drone(*this); }
};

class Fleet {
    std::vector<Robot*> robots;
public:
    Fleet() = default;
    ~Fleet() {
        for (Robot* r : robots) delete r; }
    Fleet(const Fleet& other) {
```

```
        for (const Robot* r : other.robots) robots.push_back(r->clone()); }
Fleet& operator=(const Fleet& other) {
    if (this != &other) {
        for (Robot* r : robots) delete r;
        robots.clear();
        for (const Robot* r : other.robots) robots.push_back(r->clone());
    }
    return *this; }
Fleet& add(const Robot& r) { robots.push_back(r.clone()); return *this; }
void printAll(std::ostream& os) const {
    for (const Robot* r : robots) { r->print(os); os << "\n"; } }
int assignToAll(const Mission& m) {
    int count = 0;
    for (Robot* r : robots) { if (r->assign(m)) ++count; }
    return count;
}
};
int main() {
    Fleet fleet;
    fleet.add(GroundRobot("G1", 100, 5))
        .add(Drone("D1", 100, 8))
        .printAll(std::cout);
    std::cout << fleet.assignToAll({MissionType::Rescue, 4}) << "\n";
    return 0;
}
```

Критерии за оценяване

12 точки. Максимални точки по класове:

- Коректни Mission и Robot: 4т. (9 полета)
- Коректен GroundRobot: 2т. (4 полета)
- Коректен Drone: 2т. (5 полета)
- Коректен Fleet: 4т. (9 полета)

За всяко вярно попълнено празно поле се добавят 0.5 точки до достигане на максимума в рамките на посочената част от програмата. Т.е. напълно верен Drone носи 2 точки.

Задача 2.

Задачата да се реши на езика C++.

Елементите на двоично дърво ще представяме чрез структури от вида:

```
struct node {
    node* parent; // родител
    node* left;   // ляв наследник
    node* right;  // десен наследник
    int value;    // стойност
    size_t size;  // брой елементи в дървото
```

```
node(int v,
      node* p = nullptr,
      node* l = nullptr,
      node* r = nullptr
) : value(v), parent(p), left(l),
    right(r), size(1)
{
    if(left) size += left->size;
    if(right) size += right->size;
};
```

Двоично дърво представяме чрез указател към корена му, а празното дърво – като nullptr. За корена на едно дърво задължително имаме `parent == nullptr`.

Разгледайте функциите:

```
node* foo(node* curr) {
    while (curr && curr->right)
        curr = curr->right;
    return curr;
};

node* bar(node* curr) {
    while(curr->parent &&
          curr == curr->parent->left)
    {
        curr = curr->parent;
    }
    curr = curr->parent;
    return curr;
}

void mystery(node* root) {
    node* curr = foo(root);
    while (curr) {
        std::cout << curr->value << "; ";
        curr = curr->left ? foo(curr->left)
                          : bar(curr);
    }
}
```

Да допуснем, че `T` е даденото вдясно дърво, в което има 6 елемента.

```

      45
     /  \
    17   56
   /  \
  -1   28
   /
  20
```

А) Посочете какво ще изведе функцията `mystery` на `STDOUT`, ако бъде извикана върху дървото `T`.

Отговор: _____

Б) Каква е времевата сложност на `mystery` в термините на нотацията голямо O (Big- O)? За улеснение означаваме с N броя на елементите в дървото, а с L – неговата височина.

Отговор: _____

В) Каква е външната (auxiliary) пространствена сложност на `mystery` в термините на Big- O ? Напомняме, че в тази сложност НЕ влиза размера на входните данни (в случая дървото `T`), а само допълнителната памет, която функцията заделя, за да извършва своята работа.

Отговор: _____

Г) Напишете функция `insert`, която добавя стойност `value` в двоично наредено дърво (BST) без повторения с корен `root`:

```
node* insert(node*& root, int value)
```

- Ако `value` вече се съдържа в дървото, функцията не го променя, а връща указател към вече съществуващия такъв елемент.
- В противен случай `value` се добавя и се връща указател към новосъздадения елемент.
- Ако коренът на дървото се промени, задайте подходяща стойност на `root`.
- След добавянето дървото трябва да остане BST.
- След добавянето всички `size` стойности трябва да бъдат коректни.
- Функцията трябва да бъде итеративна. Рекурсивно решение носи нула точки.
- Времевата сложност да бъде $O(L)$, където L е височината на дървото.
- Външната (auxiliary) пространствена сложност да бъде $O(1)$.
- Новият елемент да се заделя с `new`.
- При неуспех от функцията да излиза подходящо изключение, а дървото да НЕ СЕ променя.

Критерии за оценяване

А) (2 т.) За напълно коректна последователност. Намаляват се 0.5 точки ако са пропуснати разделителите.

Б) (1 т.) За коректно посочена сложност.

В) (1 т.) За коректно посочена сложност.

Г) (8 т.) За напълно коректно решение:

- (2 т.) Намиране на подходящата позиция.
- (1 т.) Ако има вече съществуващ елемент той се връща, а дървото не се променя.
- (1 т.) Добавяне на новия елемент по правилния начин, вкл. обновяване на родителя.
- (1 т.) Променя се root при промяна на корена.
- (2 т.) Коректно се обновяват size стойностите.
- (1 т.) Връщане се новодобавеният елемент (ако е било направено):
- Ако функцията е рекурсивна решението се оценява с нула точки.
- За по-лоша сложност от нужната точките се намаляват според влошаването. Ако то е сериозно цялото решение се оценява с нула точки.

Примерно решение

А) 56; 45; 28; 20; 17; -1;

Б) $O(N)$

В) $O(1)$

Г) Възможно решение е например:

```
node* insert(node*& root, int value) {
    node* parent = nullptr;
    node* curr = root;

    while (curr) {
        parent = curr;

        if (value < curr->value)
            curr = curr->left;
        else if (value > curr->value)
            curr = curr->right;
        else
            return curr;
    }

    node* n = new node(value, parent);

    if (!parent)
        root = n;
    else if (value < parent->value)
        parent->left = n;
    else
        parent->right = n;

    curr = n;
    while (curr->parent) {
        curr->parent->size++;
        curr = curr->parent;
    }

    return n;
}
```

Задача 3. Задачата да се реши на един от езиците Haskell или Scheme.

В роботизиран склад работи изпълняват задачи по транспорт на стоки. Всяка задача се представя с наредена тройка (списък от три елемента за Scheme) от идентификатор на робот (низ), тип товар (низ) и тегло на товара в кг (дробно положително число). Роботите се представят с наредени двойки (списък от два елемента за Scheme) от идентификатор (низ) и максимален капацитет на товара в кг (дробно положително число). Нека са дадени глобално дефинирани списъци: robots с роботи и tasks със задачи. В robots няма два робота с еднакъв идентификатор. Всички идентификатори на работи в tasks са валидни. Да се попълнят празните места по-долу, така че:

- robotTasks да връща списък от всички задачи на робота с идентификатор rid, чието тегло е по-голямо или равно на minWeight;
- efficiency да връща коефициента на натоварването на робота с валиден идентификатор rid, изчислен като общото тегло на товара му се раздели на максималния му капацитет;
- isLightest да проверява дали теглото на подадената задача е по-малко или равно на теглото на всяка друга задача в tasks от същия тип и за същия робот (ако няма такива да се връща истина);
- urgent да съдържа списък от всички задачи в tasks, чието тегло надвишава 40% от максималния капацитет на съответния робот, изпълняващ задачата.

Упътване: Позволено е използването на функциите assoc, elem, filter, foldr, apply, map, както и на всички функции от R⁵RS и Prelude.

Haskell

```
robotTasks rid minWeight = _____ (\_____ -> _____) tasks
efficiency rid = _____ [_____] /
                    _____ [_____]
isLightest _____ = _____ $ filter (\_____ -> _____ && _____ && _____) tasks
urgent = _____ (\_____ -> let maxCapacity = _____
                                in _____ > _____)
                                tasks
```

Пример:

```
robots = [("R1",65.5), ("R2",40.0)]
tasks = [("R1","ta",10.1), ("R1","ta",50.2), ("R1","tb",5.2), ("R2","tb",25.4), ("R2","tb",9.6)]
robotTasks "R1" 10.0 → [("R1","ta",10.1),("R1","ta",50.2)] efficiency "R1" → 1.0
isLightest ("R1","ta",10.1) → True efficiency "R2" → 0.875
isLightest ("R2","tb",25.4) → False urgent → [("R1","ta",50.2),("R2","tb",25.4)]
```

Scheme

```
(define (robotTasks rid minWeight)
  (_____ (lambda (t) _____) tasks))
(define (efficiency rid) (/ (_____ (_____))
                           (_____ (_____))))
(define (isLightest t) (_____ (filter (lambda (s) (and _____
                                                           _____
                                                           _____))
                                     tasks)))
(define urgent (_____ (lambda (t) (let ((maxCapacity _____))
                                   (> _____ _____)))
               tasks))
```

Пример:

```
(define robots '(("R1" 65.5) ("R2" 40.0)))
(define tasks '(("R1" "ta" 10.1)("R1" "ta" 50.2)("R1" "tb" 5.2)("R2" "tb" 25.4)("R2" "tb" 9.6)))
(robotTasks "R1" 10.0) → (("R1" "ta" 10.1) ("R1" "ta" 50.2)) (efficiency "R1") → 1.0
(isLightest '("R1" "ta" 10.1)) → #t (efficiency "R2") → 0.875
(isLightest '("R2" "tb" 25.4)) → #f urgent → (("R1" "ta" 50.2) ("R2" "tb" 25.4))
```

Примерно решение**Haskell**

```
robotTasks rid minWeight = filter (\(rid',_,w') -> rid' == rid && w' >= minWeight) tasks

efficiency rid = sum [w' | (rid',_,w') <- tasks, rid' == rid] /
                    head [c' | (rid',c') <- robots, rid' == rid]

isLightest (rid, t, w) = null $ filter (\(rid', t', w') -> rid == rid' && t == t' && w' < w) tasks

urgent = filter (\(rid,_,w) -> let maxCapacity = snd (head (filter (\(r,_) -> r == rid) robots))
                             in w * 5 > maxCapacity * 2)
        tasks
```

Scheme

```
(define (robotTasks rid minWeight)
  (filter (lambda (t) (and (equal? (car t) rid) (>= (caddr t) minWeight))) tasks))

(define (efficiency rid) (/ (apply + (map caddr (filter (lambda (t) (equal? (car t) rid)) tasks)))
                           (cadr (assoc rid robots))))

(define (isLightest t) (null? (filter (lambda (s) (and (equal? (car t) (car s))
                                                         (equal? (cadr t) (cadr s))
                                                         (< (caddr s) (caddr t))))
                                   tasks)))

(define urgent (filter (lambda (t) (let ((maxCapacity (cadr (assoc (car t) robots))))
                                   (> (* (caddr t) 5) (* maxCapacity 2))))
                      tasks))
```

Критерии за оценяване

Ако е писано и по двата езика, се взима по-високият резултат. Сумата от точките се закръглява до цяло число.

Максимален резултат: **12 т.**

Haskell

По ред на празните слотове:

- 1 т. за функцията `filter` или еквивалентен израз;
- не се добавят или отнемат точки, служи за означаване на параметъра на анонимната функция;
- 1 т. за напълно коректен израз, който проверява дали текущата задача отговаря на условието на `robotTasks`;
 - коректно решение на `robotTasks` на мястото на слотове от 1 до 3 се оценява с 1 т., тъй като не е спазен даденият шаблон;
- 0,5 т. за функцията `sum` или еквивалентен израз;
- 1 т. за коректен израз, който намира списък с теглата на всички задачи на робот с идентификатор `rid`;
- 1 т. за функцията `head` или еквивалентен израз;
- 1,5 т. за коректен израз, който намира капацитета на робот с идентификатор `rid`;
 - 2,5 т. при коректен израз, който намира максималния капацитет на робот с идентификатор `rid` на мястото на слотове 6 и 7;
 - коректно решение на `efficiency` на мястото на слотове от 4 до 7 се оценява с 3 т., тъй като не е спазен даденият шаблон;
- не се добавят или отнемат точки, служи за означаване и/или съпоставяне с образец на параметъра на функцията `isLightest`;
- 1 т. за функцията `null` или друга коректна проверка на филтрирания списък;
- не се добавят или отнемат точки, служи за означаване на параметъра на анонимната функция;
- 0,5 т. за проверка дали роботът на текущо разглежданата от списъка задача съвпада с работата на задачата, която е подадена като параметър на `isLightest`;
- 0,5 т. за проверка дали типът на текущо разглежданата от списъка задача съвпада точно с типа на задачата, която е подадена като параметър на `isLightest`;
- 0,5 т. за проверка дали теглото на текущо разглежданата от списъка задача е по-малко от теглото на нашата задача, която е подадена като параметър на `isLightest`;
 - приема се разместване на реда на изразите в слотове от 11 до 13, като всеки коректен израз съгласно описанието на тези слотове се оценява със съответния брой точки;
 - коректно решение на `isLightest` на мястото на слотове от 8 до 13 се оценява с 1,5 т., тъй като не е спазен даденият шаблон;
- 0,5 т. за функцията `filter` или еквивалентен израз;
- не се добавят или отнемат точки, служи за означаване на параметъра на анонимната функция;
- 2 т. за коректен израз, който намира капацитета на работата, който изпълнява текущо разглежданата задача;
- (оценява се заедно със следващия слот) 1 т. за напълно верен израз, който проверява дали теглото на текущата задача надвишава 40% от максималния капацитет на работата.
- оценява се заедно с предишния слот;
 - коректно решение за `urgent` на мястото на слотове от 14 до 18 се оценява с 2,5 т., тъй като не е спазен даденият шаблон.

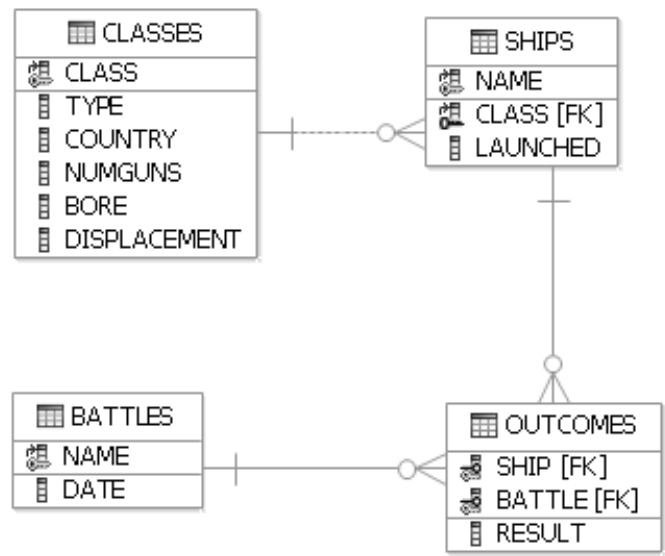
Scheme

По ред на празните слотове:

- 1 т. за функцията `filter` или еквивалентен израз;
- 1 т. за напълно коректен израз, който проверява дали текущата задача отговаря на условието на `robotTasks`;
 - коректно решение на `robotTasks` на мястото на слотове от 1 до 2 се оценява с 1 т., тъй като не е спазен даденият шаблон;
- 0,5 т. за `apply +, foldr + 0` или еквивалентен израз;
- 1 т. за коректен израз, който намира списък с теглата на всички задачи на робот с идентификатор `rid`;
- 1 т. за функцията `cadr` или еквивалентен израз;
- 1,5 т. за коректен израз, който намира подписъчка за робота с идентификатор `rid` в списъка `robots`;
 - 2,5 т. при коректен израз, който намира максималния капацитет на робот с идентификатор `rid` на мястото на слотове 5 и 6;
 - коректно решение на `efficiency` на мястото на слотове от 3 до 6 се оценява с 3 т., тъй като не е спазен даденият шаблон;
- 1 т. за функцията `null?` или друга коректна проверка на филтрирания списък;
- 0,5 т. за проверка дали роботът на текущо разглежданата от списъка задача съвпада с робота на задачата, която е подадена като параметър на `isLightest`;
- 0,5 т. за проверка дали типът на текущо разглежданата от списъка задача съвпада точно с типа на задачата, която е подадена като параметър на `isLightest`;
- 0,5 т. за проверка дали теглото на текущо разглежданата от списъка задача е по-малко от теглото на нашата задача, която е подадена като параметър на `isLightest`;
 - приема се разместване на реда на изразите в слотове от 8 до 10, като всеки коректен израз съгласно описанието на тези слотове се оценява със съответния брой точки;
 - коректно решение на `isLightest` на мястото на слотове от 7 до 10 се оценява с 1,5 т., тъй като не е спазен даденият шаблон;
- 0,5 т. за функцията `filter` или еквивалентен израз;
- 2 т. за коректен израз, който намира капацитета на робота, който изпълнява текущо разглежданата задача;
- (оценява се заедно със следващия слот) 1 т. за напълно верен израз, който проверява дали теглото на текущата задача надвишава 40% от максималния капацитет на робота.
- оценява се заедно с предишния слот;
 - коректно решение за `urgent` на мястото на слотове от 11 до 14 се оценява с 2,5 т., тъй като не е спазен даденият шаблон.

Задача 4.

Дадена е базата от данни Ships. В нея се съхранява информация за кораби (*Ships*) и тяхното участие в битки (*Battles*) по време на Втората световна война. Всеки кораб е построен по определен стереотип, определящ класа на кораба (*Classes*).



Таблицата *Classes* съдържа информация за класовете кораби:

- *class* – име на класа, първичен ключ;
- *type* – тип ('bb' за бойни кораби и 'bc' за бойни крайцери);
- *country* – държавата, която строи такива кораби;

- *numGuns* – броят на основните оръдия;
- *bore* – калибъра им (диаметърът на отвора на оръдието в инчове);
- *displacement* – водоизместимост (в тонове).

Таблицата *Ships* съдържа информация за корабите:

- *name* – име на кораб, първичен ключ;
- *class* – име на неговия клас;
- *launched* – годината, в която корабът е пуснат на вода.

Таблицата *Battles* съдържа информация за битките:

- *name* – име на битката, първичен ключ;
- *date* – дата на провеждане.

Таблицата *Outcomes* съдържа информация за резултатата от участието на даден кораб в дадена битка. Колоните *ship* и *battle* заедно формират първичния ключ.

- *ship* – име на кораба;
- *battle* – име на битката;
- *result* – резултат (потънал – 'sunk', повреден – 'damaged', победил – 'ok').

1) Да се напише SELECT заявка, която за всеки кораб, който има по-малка водоизместимост от средната водоизместимост на класовете на същата държава като неговата, извежда следната информация:

- име на кораба;
- година на пускане на вода;
- брой участия в битки на кораба (или числото 0, ако корабът не е участвал в нито една битка).

Резултатът да бъде сортиран така, че първо да се извеждат най-новите кораби.

2) Дадена е релацията *R* със следната схема: *R*(*A*, *B*, *C*, *D*). Домейните на атрибутите са атомарни множества. В сила са следните функционални зависимости:

$$AB \rightarrow C$$

$$AB \rightarrow D$$

$$C \rightarrow B$$

Намерете всички множества от атрибути, които са ключове на *R*. Обосновайте решението си.

Примерно решение

1)

```
SELECT name, launched, COUNT(battle) AS battles
FROM Ships s
JOIN Classes c ON c.class = s.class
LEFT JOIN Outcomes o ON s.name = o.ship
WHERE displacement < (SELECT AVG(displacement)
                      FROM Classes
                      WHERE country = c.country)
GROUP BY name, launched
ORDER BY launched DESC;
```

2)

Тъй като атрибутът A не присъства в дясната страна на нито една функционална зависимост (ΦZ), то всеки ключ съдържа A .

Тъй като атрибутът D не присъства в лявата страна на нито една ΦZ , но присъства в дясната страна на една от тях, то нито един ключ не съдържа D .

Следователно трябва последователно да проверим за всяко от множествата $\{A\}$, $\{A, B\}$, $\{A, C\}$ и $\{A, B, C\}$. Множеството от атрибути X е суперключ на R , ако $X^+ = \{A, B, C, D\}$. X е ключ, ако е суперключ и никое негово собствено подмножество не е суперключ.

$\{A\}$ не е суперключ, тъй като $\{A\}^+ = \{A\}$, следователно не е и ключ.

$\{A, B\}^+ = \{A, B, C, D\}$, т.е. $\{A, B\}$ е суперключ. Вече беше доказано, че $\{A\}$ не е ключ, аналогично и $\{B\}$ не е ключ, понеже $\{B\}^+ = \{B\}$. Следователно $\{A, B\}$ е ключ.

Аналогично намираме $\{A, C\}^+ = \{A, B, C, D\}$ и показваме, че $\{C\}^+ = \{B, C\}$, т.е. $\{A, C\}$ също е ключ.

След като $\{A, B\}$ е ключ, то $\{A, B, C\}$ е суперключ, но не е ключ.

Критерии за оценяване

1) Коректно решение носи 6 т. За всяка сгрешена клауза броят на точките се намалява с 2 до достигане на 0 т.

2) Посочването на верните отговори носи до 2 т., а доказателството им носи още до 4 т. по следната схема:

Всички ключове на релацията са $\{A, B\}$ и $\{A, C\}$. Посочването на всеки от тези отговори носи по 1 т., а при посочването на грешно множество от атрибути броят на точките се намалява с по 1 за всеки грешен отговор до достигане на 0 т.

Коректно доказателство за всеки от двата ключа, използващо обвивки на атрибути или аксиоми на Армстронг, носи по 2 точки.

Задача 5.

Казваме, че едно множество $W \subseteq \{0, 1\}^*$ е *отворено*, ако съществува множество $U \subseteq \{0, 1\}^*$ и крайно множество $V \subseteq \{0, 1\}^*$, такива че

$$W = V \cup (U \cdot \{0, 1\}^*).$$

Докажете, че ако L и $\bar{L}$ са отворени, то L е регулярен.

Примерно решение

Означаваме азбуката $\{0, 1\}$ със Σ . Съгласно условието на задачата $L = V_L \cup U_L \cdot \Sigma^*$ и $\bar{L} = V_{\bar{L}} \cup U_{\bar{L}} \cdot \Sigma^*$, за някои $U_L, U_{\bar{L}}, V_L, V_{\bar{L}} \subseteq \Sigma^*$, $V_L, V_{\bar{L}}$ – крайни. Да фиксираме $m_0 > 0$, такова че $V_L, V_{\bar{L}} \subseteq \{w \in \Sigma^* \mid |w| < m_0\} = \Sigma^{<m_0}$ и нека

$$L' = (L \cap \Sigma^{<m_0}) \cup (L \cap \Sigma^{m_0}) \cdot \Sigma^*.$$

Ще докажем, че $L = L'$ (4 т.). За думи w с $|w| < m_0$ е ясно, че $w \in L \iff w \in L'$, за това нека $|w| \geq m_0$ и $w = u_w \cdot x$, където $|u_w| = m_0$. Нека първо $w \in L$. Да допуснем, че $w \notin L'$. Тогава $u_w \notin L$, т.е. $u_w \in \bar{L}$. Съгласно избора на m_0 , $u_w \notin V_{\bar{L}}$ и следователно за някои $u \in U_{\bar{L}}$ и $y \in \Sigma^*$, $u_w = u \cdot y$, откъдето $w = u \cdot y \cdot x$ (защото $w = u_w \cdot x$) и значи $w \in U_{\bar{L}} \cdot \Sigma^* \subseteq \bar{L}$, противоречие. Следователно $L \subseteq L'$ (3 т.). Обратно, ако $w \in L'$, то $u_w \in L$ и следователно за някои $u \in U_L$ и $y \in \Sigma^*$, $u_w = u \cdot y$, (защото $u_w \notin V_L$), откъдето $w = u \cdot y \cdot x$ и значи $w \in U_L \cdot \Sigma^* \subseteq L$. Следователно $L' \subseteq L$ (3 т.).

Тъй като $L \cap \Sigma^{<m_0}$ и $L \cap \Sigma^{m_0}$ са крайни, а значи и регулярни, то $L = L'$ е регулярен (2 т.).

Критерии за оценяване

Съобразно посочените етапи на решението.

Задача 6.

Да се пресметне интегралът

$$\int_1^{\sqrt{e}} \frac{\arccos(\ln x)}{x} dx$$

Примерно решение
 Преобразуваме последователно:

$$\int_1^{\sqrt{e}} \frac{\arccos(\ln x)}{x} dx =$$

(1 т.)

$$= \int_1^{\sqrt{e}} \arccos(\ln x) d \ln x =$$

(3 т.)

(полагаме $\ln x = t : x = 1 \iff t = 0; x = \sqrt{e} \iff t = 1/2$)

$$= \int_0^{\frac{1}{2}} \arccos(t) dt =$$

(2 т.)

(интегрираме по части)

$$= t \arccos(t) \Big|_0^{\frac{1}{2}} - \int_0^{\frac{1}{2}} t d \arccos(t) =$$

(1 т. + 1 т.)

$$= \frac{1}{2} \arccos\left(\frac{1}{2}\right) - 0 - \int_0^{\frac{1}{2}} \frac{-t}{\sqrt{1-t^2}} dt =$$

(1 т. + 1 т.)

$$= \frac{\pi}{6} - \frac{1}{2} \int_0^{\frac{1}{2}} \frac{1}{\sqrt{1-t^2}} d(1-t^2) =$$

(1 т.)

$$= \frac{\pi}{6} - \sqrt{1-t^2} \Big|_0^{\frac{1}{2}}$$

(1 т.)

$$= \frac{\pi}{6} + 1 - \frac{\sqrt{3}}{2}$$

Критерии за оценяване

Съобразно посочените етапи на решението.

Чернова